



Seminario de Programación



Lenguaje C: Manejo de Memoria y elementos con punteros



Patricio Olivares

Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

1. Cómo se Organiza la Memoria en C

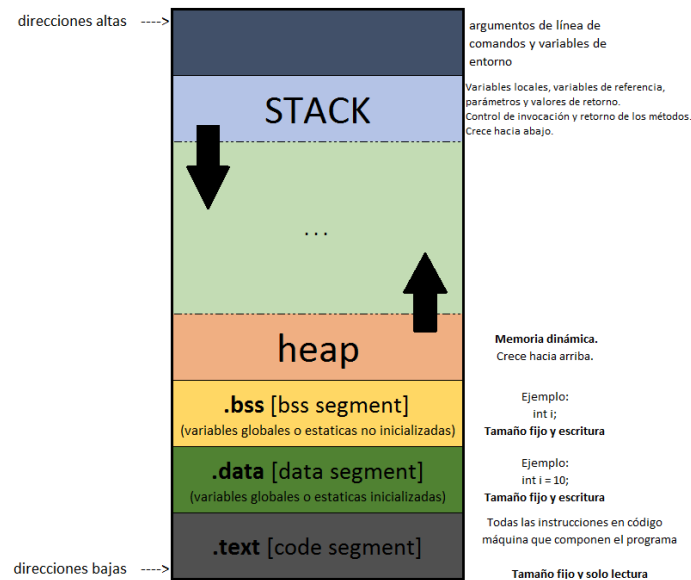
Cuando un programa en **C** se ejecuta, el sistema operativo le asigna un espacio de memoria. Este espacio se divide en **segmentos** que cumplen diferentes funciones:

Segmentos principales

- **Segmento de código o texto (code)** Contiene el **código ejecutable** del programa (instrucciones compiladas). Es de solo lectura para evitar modificaciones en tiempo de ejecución.
- **Segmento estático (static)** Almacena **variables globales y estáticas**, incluyendo nombres de funciones. Realiza ligado estático de variables en memoria durante toda la ejecución. Se divide en dos:
 - Variables inicializadas.
 - Variables no inicializadas (rellenadas con 0).

- **Segmento o memoria heap** Segmento donde se le permite al **programador** reservar memoria de forma dinámica. Se usa mediante **punteros**. El programador debe liberar lo que reserva (`malloc` , `calloc` , `realloc` , `free`).
- **Segmento o memoria stack** Área dinámica de memoria que almacena **parámetros de funciones, variables locales y frames de ejecución**. La administración es **automática por el compilador** (se reserva al entrar en una función y se libera al salir).

Diagrama simplificado



1.1 Segmento de código o texto (code)

El **segmento de código** contiene las **instrucciones compiladas** del programa en C. Aquí se almacenan las funciones (`main` , otras funciones definidas por el programador, y funciones externas enlazadas).

- Es **solo lectura**: no se puede modificar en tiempo de ejecución.
- Cada función tiene una dirección fija dentro del binario cargado en memoria.

Ejemplo en C:

```
#include <stdio.h>

// --- STATIC (segmento estático) ---
int a = 10;           // global inicializada (static init)
float pi = 3.1415f;  // global inicializada (static init)

// --- CODE (segmento de código) ---
void func(void) {
    // función para ubicar su dirección en el segmento de código
}
```

```

}

int main(void) {
    // --- STACK (segmento stack) ---
    int b;        // variable local en stack
    b = 45;       // como en los diagramas

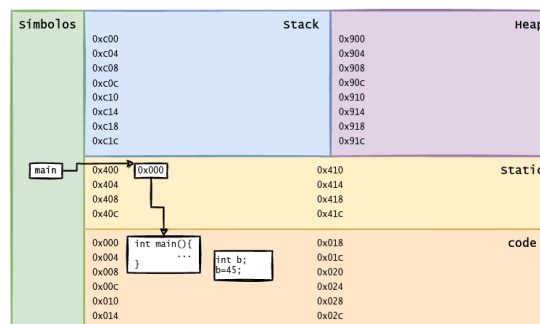
    printf("Direccion de main (code): %p\n", (void*)main);
    printf("Direccion de func (code): %p\n", (void*)func);
    printf("Direccion de a (static): %p, valor a = %d\n",
    (void*)&a, a);
    printf("Direccion de pi (static): %p, valor pi = %f\n",
    (void*)&pi, pi);
    printf("Direccion de b (stack): %p, valor b = %d\n", (void*)&b,
    b);

    return 0;
}

```

Este código imprime las direcciones en memoria donde están almacenadas las funciones. Corresponden al **segmento de código**.

El especificador `%p` en `printf` (`p` = pointer) se usa para imprimir direcciones de memoria en formato hexadecimal, y por convención se recomienda castear a `(void*)` para mostrar correctamente cualquier tipo de puntero.*



1.2 Segmento estático (static)

El **segmento estático** almacena:

- **Variables globales** (visibles en todo el programa).
- **Variables estáticas** (conservan su valor entre ejecuciones de funciones).
- **Constantes globales**.

Este segmento se divide en:

- Variables inicializadas (ej: `int a = 10;`).
- Variables no inicializadas (ej: `int a;` → se llena con 0).

Ejemplo en C:

```

#include <stdio.h>

// --- STATIC (segmento estático) ---
int a = 10;           // global inicializada (static init)
float pi = 3.1415f;   // global inicializada (static init)

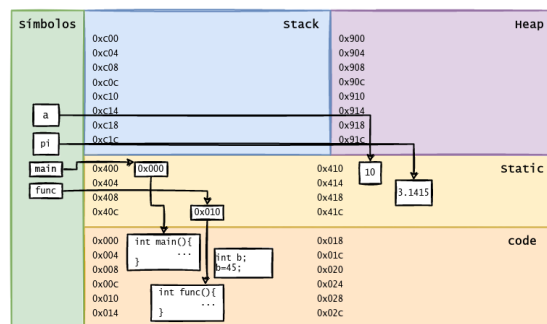
// --- CODE (segmento de código) ---
void func(void) {
    // función para ubicar su dirección en el segmento de código
}

int main(void) {
    // --- STACK (segmento stack) ---
    int b;           // variable local en stack
    b = 45;          // como en los diagramas

    printf("Direccion de main (code): %p\n", (void*)main);
    printf("Direccion de func (code): %p\n", (void*)func);
    printf("Direccion de a (static): %p, valor a = %d\n",
    (void*)&a, a);
    printf("Direccion de pi (static): %p, valor pi = %f\n",
    (void*)&pi, pi);
    printf("Direccion de b (stack): %p, valor b = %d\n", (void*)&b,
    b);

    return 0;
}

```



1.3 Segmento o memoria stack

El **stack** se utiliza para:

- Variables locales (definidas dentro de funciones).
- Parámetros de funciones.
- Control de flujo (información de llamadas, retorno, etc).

Características:

- Se maneja **automáticamente**: se reserva al entrar en una función y se libera al salir.
- Permite recursión, ya que cada llamada genera un nuevo *stack frame*.

Ejemplo en C:

```
#include <stdio.h>

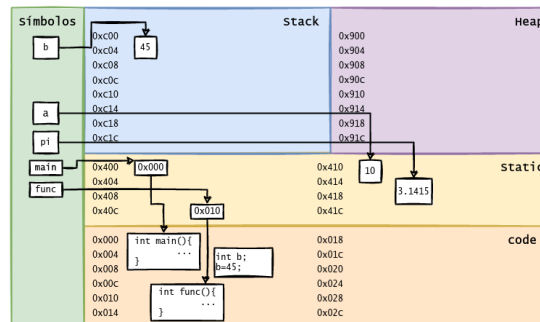
// --- STATIC (segmento estático) ---
int a = 10;           // global inicializada (static init)
float pi = 3.1415f;   // global inicializada (static init)

// --- CODE (segmento de código) ---
void func(void) {
    // función para ubicar su dirección en el segmento de código
}

int main(void) {
    // --- STACK (segmento stack) ---
    int b;           // variable local en stack
    b = 45;          // como en los diagramas

    printf("Direccion de main (code): %p\n", (void*)main);
    printf("Direccion de func (code): %p\n", (void*)func);
    printf("Direccion de a (static): %p, valor a = %d\n",
    (void*)&a, a);
    printf("Direccion de pi (static): %p, valor pi = %f\n",
    (void*)&pi, pi);
    printf("Direccion de b (stack): %p, valor b = %d\n", (void*)&b,
    b);

    return 0;
}
```



1.4 Segmento o memoria Heap

Qué es el heap (visión general). El *heap* es la zona de memoria destinada a **asignaciones dinámicas** en tiempo de ejecución. A diferencia del *stack* (que se gestiona automáticamente al entrar/salir de una función), en el heap **tú decides cuándo reservar y liberar** memoria. Si la reservas y no la liberas, se produce una **fuga de memoria** (*memory leak*). El heap es ideal cuando el tamaño o la vida útil de los datos **no se conoce** en tiempo de compilación.

Punteros en C Un **puntero** es una variable que **almacena direcciones de memoria**.

- `&x` → "dirección de `x`" (operador **address-of/referencia**).
- `*p` → "valor apuntado por `p`" (operador **dereferencia**).
- Los punteros tienen **tipo**: `int *` apunta a `int`, `float *` a `float`, etc.
- Para imprimir direcciones usamos `%p` (y casteamos a `(void*)`).

Heap usando punteros (cómo se conectan). Para usar el heap necesitas un **puntero** que guarde la dirección del bloque reservado:

1. Declaras el puntero (en el **stack**).
2. Reservas en el **heap** con `malloc` / `calloc` / `realloc`, que devuelve una **dirección**.
3. Usas esa memoria vía `*p` o `p[i]`.
4. **Libera** con `free(p)`

Ejemplo en C:

```
#include <stdio.h>
#include <stdlib.h>    // malloc, free

// --- STATIC (segmento estático) ---
int a = 10;           // global inicializada
float pi = 3.1415f;   // global inicializada

// --- CODE (segmento de código) ---
void func(void) {
    // cuerpo vacío: nos interesa su dirección en el segmento de
    código
}

int main(void) {
    // --- STACK (segmento stack) ---
    int b = 45;        // variable local en stack

    // --- HEAP (asignación dinámica usando puntero) ---
    int *c = (int *) malloc(sizeof(int)); // reserva de 1 entero
    en heap
    if (!c) {
        perror("malloc");
        return 1;
    }
    *c = 100;          // usar la memoria dinámica

    printf("Direccion de main (code): %p\n", (void*)main);
    printf("Direccion de func (code): %p\n", (void*)func);

    printf("Direccion de a (static): %p, a = %d\n", (void*)&a,
a);
    printf("Direccion de pi (static): %p, pi = %f\n", (void*)&pi,
pi);
```

```

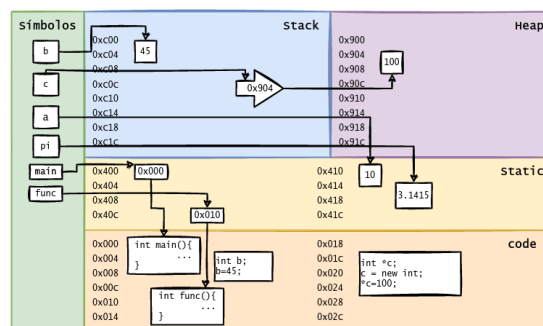
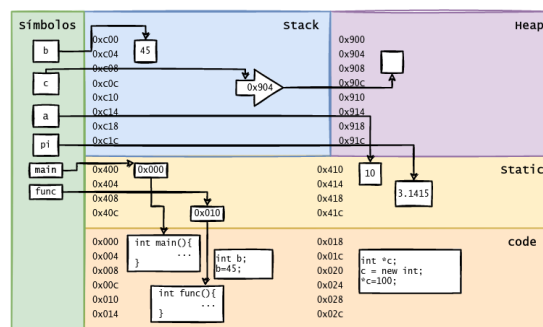
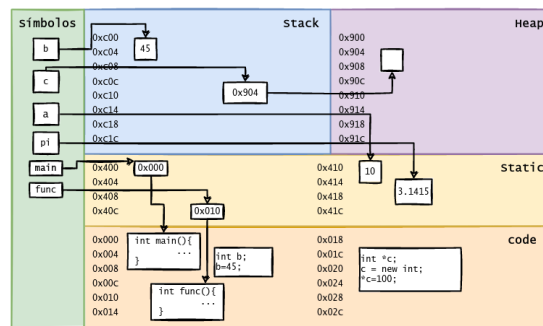
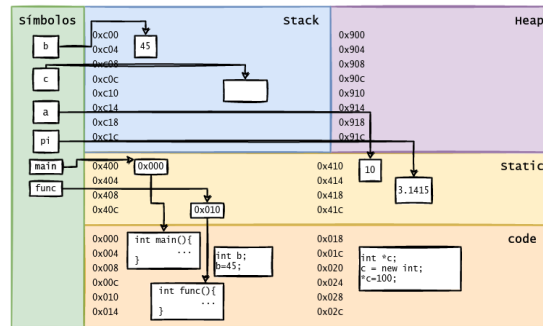
printf("Direccion de b      (stack):  %p, b = %d\n", (void*)&b,
b);

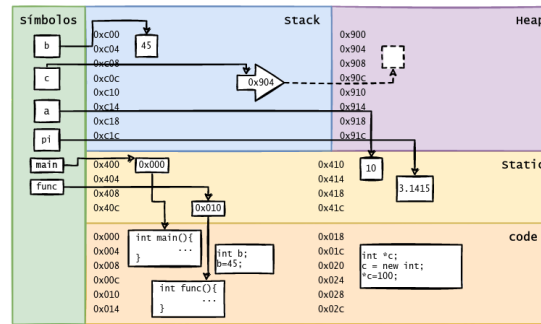
printf("Direccion de c      (stack, puntero): %p\n", (void*)&c);
printf("Valor de c (direccion en heap):      %p\n", (void*)c);
printf("Contenido en heap (*c):              %d\n", *c);

free(c);    // liberar la memoria del heap

return 0;
}

```





Funciones principales para administrar memoria dinámica en C

1. malloc

`malloc` reserva un bloque de memoria en el **heap** del tamaño indicado. El contenido queda sin inicializar. Devuelve un puntero genérico (`void*`) al inicio del bloque, o `NULL` si la reserva falla.

Encabezado:

```
void *malloc(size_t size);
```

- `size` : número de bytes a reservar.
- `void*` : puntero genérico que normalmente se convierte al tipo deseado (ej: `int*`, `float*`).

Ejemplo:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = (int*)malloc(5 * sizeof(int)); // reserva espacio
    para 5 enteros
    if (arr == NULL) {
        perror("malloc");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        arr[i] = i + 1;
        printf("arr[%d] = %d\n", i, arr[i]);
    }

    free(arr);
    return 0;
}
```


2. calloc

`calloc` reserva memoria para varios elementos de un mismo tipo y los inicializa en cero. Es útil cuando se necesita asegurar que la memoria esté limpia.

Encabezado:

```
void *calloc(size_t num, size_t size);
```

- `num`: cantidad de elementos.
- `size`: tamaño en bytes de cada elemento.
- `void*`: puntero genérico al inicio del bloque reservado.

Ejemplo:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = (int*)calloc(5, sizeof(int)); // 5 enteros
    inicializados en 0
    if (arr == NULL) {
        perror("calloc");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        printf("arr[%d] = %d\n", i, arr[i]); // imprime 0
    }

    free(arr);
    return 0;
}
```

3. realloc

`realloc` permite redimensionar un bloque de memoria previamente reservado, ajustando su tamaño a uno nuevo. Si es necesario, puede mover el bloque a otra dirección, conservando el contenido ya existente hasta el límite del tamaño menor (entre el original y el nuevo).

Encabezado:

```
void *realloc(void *ptr, size_t new_size);
```

- `ptr`: puntero al bloque previamente reservado (con `malloc` o `calloc`).
- `new_size`: nuevo tamaño en bytes.
- Devuelve un puntero al bloque redimensionado o `NULL` si falla.

Ejemplo:

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = (int*)malloc(3 * sizeof(int));
    if (arr == NULL) return 1;

    for (int i = 0; i < 3; i++) arr[i] = i + 1;

    arr = (int*)realloc(arr, 6 * sizeof(int)); // ampliar a 6
    enteros
    if (arr == NULL) return 1;

    for (int i = 3; i < 6; i++) arr[i] = (i + 1) * 10;

    for (int i = 0; i < 6; i++)
        printf("arr[%d] = %d\n", i, arr[i]);

    free(arr);
    return 0;
}

```

4. free

`free` libera un bloque de memoria reservado previamente con `malloc`, `calloc` o `realloc`. No devuelve nada. El puntero sigue existiendo pero queda "suelto/colgando/dangling", por lo que es buena práctica asignarle `NULL`.

Encabezado:

```
void free(void *ptr);
```

- `ptr`: puntero al bloque que se desea liberar.

Ejemplo:

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p = (int*)malloc(sizeof(int));
    if (p == NULL) return 1;

    *p = 99;
    printf("Valor en heap: %d\n", *p);

    free(p); // liberar memoria
    p = NULL; // buena práctica

    return 0;
}

```

2. Errores comunes en el manejo de memoria

2.1 Fuga de memoria (*memory leak*)

Una fuga ocurre cuando se **pierde la única referencia** a un bloque del heap: se reserva memoria y luego se reasigna el puntero sin **liberar** primero el bloque anterior. Ese bloque queda inaccesible hasta que el proceso termina (no hay forma de recuperarlo).

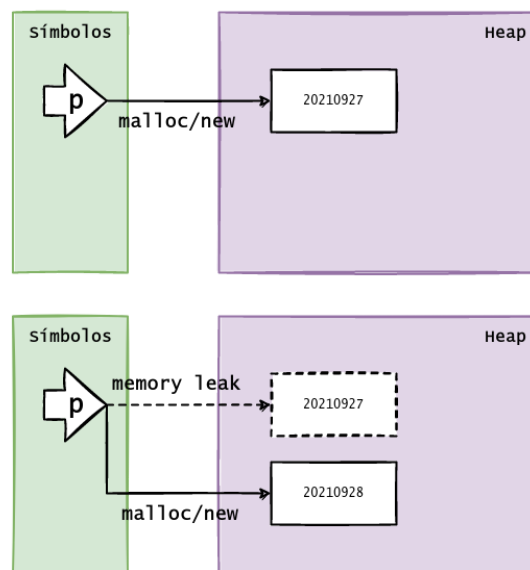
Programa con fuga

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *p = (int*) malloc(sizeof(int));    // reserva #1
    if (!p) return 1;
    *p = 20210927;
    printf("p -> %p, *p = %d (bloque 1)\n", (void*)p, *p);

    // Se necesita "otro" entero en heap, pero se OLVIDA liberar el
    // primero:
    p = (int*) malloc(sizeof(int));        // reserva #2 (FUGA
    // del bloque 1)
    if (!p) return 1;
    *p = 20210928;
    printf("p -> %p, *p = %d (bloque 2)\n", (void*)p, *p);

    // Solo se libera el bloque 2; el bloque 1 quedó perdido
    free(p);
    return 0;
}
```



Versión correcta (liberar antes de reasignar):

```

#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *p = (int*) malloc(sizeof(int));
    if (!p) return 1;
    *p = 20210927;
    printf("p -> %p, *p = %d (bloque 1)\n", (void*)p, *p);

    free(p);          // 1) liberar el bloque actual
    p = NULL;         // 2) anular la referencia (buena práctica)

    p = (int*) malloc(sizeof(int)); // 3) nueva reserva segura
    if (!p) return 1;
    *p = 20210928;
    printf("p -> %p, *p = %d (bloque 2)\n", (void*)p, *p);

    free(p);
    p = NULL;
    return 0;
}

```

2.2 Puntero colgante (*dangling pointer*)

Un puntero queda **colgante** cuando apunta a memoria que ya fue **liberada** (o salió de alcance). Caso típico: dos punteros (alias) apuntan al mismo bloque; se libera con uno y luego se usa el otro.

Programa que genera

```

#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *p = (int *) malloc(sizeof(int));
    if (!p) return 1;
    *p = 123;

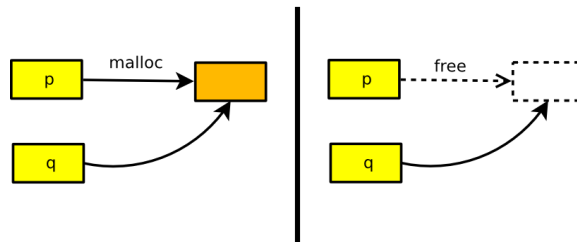
    int *q = p; // q es un alias: p y q apuntan al MISMO bloque
    printf("p = %p, q = %p, *p = %d\n", (void*)p, (void*)q, *p);

    free(p);    // se libera el bloque al que apuntaban p y q

    // A partir de aquí, q (y p) están COLGANDO: no deben usarse
    *q = 456;    // UAF: comportamiento indefinido
    printf("%d\n", *q); // también indefinido

    return 0;
}

```



Formas seguras de evitarlo:

- **Invalidar todos los alias** cuando compartan la misma propiedad del bloque:

```
#include <stdlib.h>
```

```
int main(void) {
    int *p = (int*) malloc(sizeof(int));
    if (!p) return 1;
    int *q = p;    // alias

    free(p);       // liberar una sola vez
    p = NULL;
    q = NULL;      // invalidar alias para evitar uso accidental
    return 0;
}
```

- **Clonar a otro bloque** si ambos punteros deben sobrevivir de forma independiente:

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main(void) {
    int *p = (int*) malloc(sizeof(int));
    if (!p) return 1;
    *p = 123;

    int *q = (int*) malloc(sizeof(int));    // bloque propio para q
    if (!q) { free(p); return 1; }
    *q = *p;                                // copiar contenido

    free(p);    // ya no afecta a q
    p = NULL;

    printf("*q = %d (q sigue siendo válido)\n", *q);

    free(q);
    q = NULL;
    return 0;
}
```

3. Buenas prácticas de gestión de memoria

3.1 Patrón reservar → usar → liberar

Siempre libera la memoria en el mismo flujo en que la usas.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p = (int*) malloc(sizeof(int));
    if (!p) return 1;
    *p = 42;
    printf("%d\n", *p);
    free(p); // liberar siempre
    p = NULL; // Buena práctica
    return 0;
}
```

Por qué importa: evita fugas de memoria (*memory leak*).

3.2 Comprobar NULL tras reservar

Nunca uses un puntero sin verificar que la reserva fue exitosa.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p = (int*) malloc(sizeof(int));
    if (!p) { // Equivalente a p == NULL
        perror("malloc");
        return 1;
    }
    free(p);
    p = NULL;
    return 0;
}
```

Por qué importa: previene usar punteros inválidos.

3.3 Evitar punteros colgantes

Tras liberar, asigna NULL para no dejar referencias sueltas.

```
#include <stdlib.h>

int main() {
    int *p = (int*) malloc(sizeof(int));
    if (!p) return 1;
    free(p);
    p = NULL; // buena práctica
    return 0;
}
```

Por qué importa: reduce riesgos de *dangling pointer*.

4. Depuración de memoria con Valgrind

4.1 ¿Qué es?

[Valgrind](#) ejecuta tu programa en un entorno controlado que detecta **fugas de memoria** (*memory leaks*) y **usos inválidos** (*dangling pointers*).

4.2 Instalación

En la mayoría de distribuciones Linux está en los repositorios oficiales:

- **Debian/Ubuntu**

```
sudo apt update
sudo apt install valgrind
```

4.3 Compilar para usar Valgrind

Compila con **símbolos de depuración** (`-g`) y sin optimizaciones (`-O0`):

```
gcc -std=c11 -g -O0 archivo.c -o app
```

- `-g` : agrega información de líneas al binario (Valgrind mostrará en qué línea ocurrió el error).
- `-O0` : desactiva optimizaciones que podrían confundir el análisis.

Por qué importa: sin estos flags, Valgrind solo mostrará direcciones de memoria, no líneas de tu código.

4.4 Ejemplo: *memory leak*

```
#include <stdlib.h>
int main() {
    int *p = (int*) malloc(10 * sizeof(int)); // no liberado
    p[0] = 1;
    return 0;
}
```

```
valgrind --leak-check=full ./app
```

Reporte típico: `definitely lost: 40 bytes in 1 blocks` → fuga confirmada.

4.5 Ejemplo: *dangling pointer*

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *p = (int*) malloc(sizeof(int));
```

```

    *p = 5;
    free(p);
    printf("%d\n", *p); // uso inválido
    return 0;
}

```

valgrind ./app

Reporte típico: Invalid read of size 4 → uso de memoria ya liberada.

5. Uso de punteros en arreglos y estructuras

En esta sección exploraremos cómo los punteros permiten trabajar con arreglos dinámicos y con estructuras en C. La meta es comprender lo esencial con ejemplos cortos y claros.

5.1 Arreglos dinámicos con `malloc` y `realloc` (push/pop simple)

En C no existen vectores que crezcan automáticamente. Con `malloc` reservamos memoria inicial y con `realloc` podemos agrandar o achicar el arreglo, conservando los valores anteriores.

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 3;
    int *v = (int*) malloc(n * sizeof(int));

    // Aumentamos el tamaño del arreglo
    n++;
    v = (int*) realloc(v, n * sizeof(int)); // crecer el arreglo
    v[n-1] = 10;

    printf("%d\n", v[n-1]); // muestra 10
    free(v);
    v = NULL;
    return 0;
}

```

5.2 Punteros y aritmética vs indexación

El acceso con índices `p[i]` es simplemente una forma más legible de escribir `*(p+i)`. Ambas expresiones hacen lo mismo: avanzar `i` posiciones desde `p` y luego desreferenciar.

```

#include <stdio.h>
#include <stdlib.h>

```



```

int main() {
    int *p = (int*) malloc(3 * sizeof(int));
    p[0] = 5; p[1] = 6;

    printf("%d\n", p[1]);    // indexación
    printf("%d\n", *(p+1)); // aritmética de punteros

    free(p);
    p = NULL;
    return 0;
}

```

5.3 struct con campos dinámicos (crear, clonar, liberar)

Un `struct` puede contener punteros. En ese caso debemos reservar memoria para los campos dinámicos y liberarlos después. Si no lo hacemos, se producen fugas de memoria.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char *nombre;
    int edad;
} Persona;

int main() {
    Persona p;
    p.nombre = (char*) malloc(5);    // reserva para la cadena.
    // malloc(5) equivale a malloc(5*sizeof(char)) porque
    // sizeof(char) es 1(byte)
    strcpy(p.nombre, "Ana");
    p.edad = 20;

    printf("%s (%d)\n", p.nombre, p.edad);

    free(p.nombre);    // liberar campo dinámico
    p.nombre = NULL;
    return 0;
}

```

5.4 Punteros a struct y operador ->

Cuando tenemos un puntero a estructura usamos el operador `->`. Es equivalente a escribir `(*ptr).campo`, pero más claro y cómodo.

```

#include <stdio.h>
#include <stdlib.h>

```

```
typedef struct {
    int edad;
} Persona;

int main() {
    Persona *p = (Persona*) malloc(sizeof(Persona));
    p->edad = 25; // igual que (*p).edad
    printf("%d\n", p->edad);

    free(p);
    p = NULL;
    return 0;
}
```

6. Ejercicios

Ejercicio 1. Preguntas teóricas de repaso

Responde de forma breve y precisa.

1. Segmentos de memoria: describe propósito y ejemplos de variables en code, static, stack y heap.
2. `%p` y `(void*)`: ¿por qué casteamos a `(void*)` al imprimir direcciones con `%p`?
3. Diferencia entre `malloc`, `calloc` y `realloc`. Indica un caso de uso típico de cada uno.
4. ¿Cuándo ocurre un memory leak? Da un ejemplo conceptual.
5. ¿Qué es un dangling pointer? Menciona dos formas de evitarlo.
6. Buenas prácticas mínimas: lista cuatro que aplicarías siempre al usar memoria dinámica.
7. Valgrind: ¿qué detecta y por qué se recomienda compilar con `-g -O0` antes de usarlo?

Respuestas en 1 a 3 líneas por pregunta.

Ejercicio 2. Inventario dinámico de biblioteca con punteros

Contexto: Una biblioteca quiere digitalizar un inventario muy simple. Necesitan:

1. guardar el número de libros disponibles en una categoría,
2. registrar los códigos de identificación de algunos ejemplares,
3. manejar información básica de una persona responsable (nombre y edad).

El sistema debe construirse usando **punteros**, sin usar indexación `[]`.

Instrucciones:**1. Variable con puntero**

- Reserva dinámicamente un entero `int* total_libros`.
- Asigna el valor `42` con `*total_libros = 42` para representar la cantidad de libros en una categoría.
- Imprime el valor con `printf`.
- Libera la memoria.

2. Arreglo dinámico con aritmética de punteros

- Reserva memoria para 3 enteros con `malloc` para guardar tres códigos de libros (ejemplo: 101, 102, 103).
- Asigna los valores usando solo `*(p+i)` y nunca `p[i]`.
- Imprime el segundo código usando `*(p+1)`.
- Libera la memoria.

3. Estructura con campo dinámico y acceso con punteros

- Define la estructura:

```
typedef struct {
    char* nombre;
    int edad;
} Persona;
```

- Reserva en heap una `Persona* encargado`.
- Reserva memoria dinámica para `nombre`, copia un texto corto (ej. "Ana") con `strcpy`, y asigna la edad.
- Imprime la información usando el operador `->`.
- Libera primero `nombre` y luego la `Persona`.

Restricciones:

- No usar `[]`. Todas las operaciones sobre arreglos deben realizarse con aritmética de punteros.
- Acceso a `struct` solo mediante `->` o `(*ptr).campo`.

Ejercicio 3. Detección y corrección de errores con Valgrind

Objetivo: analizar un programa con Valgrind, identificar errores de memoria y corregirlos manteniendo la funcionalidad solicitada y el uso de memoria dinámica.

Funcionalidad requerida:

- Crear un arreglo dinámico de `n` enteros.
- Rellenarlo con 1, 2, ..., n.
- Calcular el promedio y mostrarlo.
- No filtrar entradas ni agregar interacción extra.

Código entregado (intencionalmente defectuoso):

```
// avg_buggy.c
#include <stdio.h>
#include <stdlib.h>

double promedio(int *a, size_t n) {
    int *tmp = a;           // alias al mismo bloque
    long suma = 0;
    for (size_t i = 0; i <= n; i++) { // error: acceso fuera de
rango
        suma += *(tmp + i);
    }
    free(a);                // libera aquí, pero el llamador
también intentará usarlo
    return (double)suma / (double)n;
}

int main(void) {
    size_t n = 5;
    int *v = malloc(n * sizeof(int));
    if (!v) return 1;

    // intento de redimensionar antes de usar, se pierde el puntero
si falla
    v = realloc(v, (n + 1) * sizeof(int));
    // sin verificar NULL

    for (size_t i = 0; i < n; i++) {
        v[i] = (int)(i + 1);
    }

    double p = promedio(v, n); // promedio libera v por dentro
// uso después de free: v ya no es válido si promedio lo liberó
printf("Primero: %d\n", v[0]); // acceso inválido

    // doble free potencial si no se alcanzó la línea anterior
    free(v);
    printf("Promedio = %.2f\n", p);
    return 0;
}
```

Tareas:

1. Compilación y análisis con Valgrind

- Analiza el siguiente comando de compilación y explica cada uno de los flags (opciones): `gcc -std=c11 -Wall -Wextra -g -O0 avg_buggy.c -o avg_buggy`
- Compila con: `gcc -std=c11 -Wall -Wextra -g -O0 avg_buggy.c -o avg_buggy`
- Ejecuta con: `valgrind --leak-check=full --show-leak-kinds=all ./avg_buggy`
- Registra los hallazgos relevantes: accesos inválidos, doble free, fugas, punteros colgantes o pérdida del puntero en `realloc`.

2. Correcciones obligatorias manteniendo la funcionalidad

- Evitar acceso fuera de rango en el bucle de suma.
- No liberar dentro de `promedio`. La propiedad del arreglo debe quedar en `main`.
- Proteger el uso de `realloc` con un temporal.
- Eliminar el uso de `v` después de que haya sido liberado.
- Asegurar una única liberación en el lugar correcto.
- Conservar la salida final: imprimir el primer elemento y el promedio calculado.

3. Validación

- Recompila la versión corregida.
- Vuelve a correr Valgrind y verifica que no existan errores ni fugas.
- Entrega el binario funcionando y el reporte de Valgrind limpio.